From the caller's side, a message that's processed by method_missing looks like a regular call—but on the receiver's side, it has no corresponding method. This trick is called a *Ghost Method*. Let's look at some Ghost Method examples.

Spell: Ghost Method

The Hashie Example

The Hashie gem contains a little bit of magic called Hashie::Mash. A Mash is a more powerful version of Ruby's standard OpenStruct class: a hash-like object whose attributes work like Ruby variables. If you want a new attribute, just assign a value to the attribute, and it will spring into existence:

```
require 'hashie'

icecream = Hashie::Mash.new
icecream.flavor = "strawberry"
icecream.flavor                 # => "strawberry"
```

This works because Hashie::Mash is a subclass of Ruby's Hash, and its attributes are actually Ghost Methods, as a quick look at Hashie::Mash.method_missing will confirm:

gems/hashie-1.2.0/lib/hashie/mash.rb

```
module Hashie
  class Mash < Hashie::Hash
    def method_missing(method_name, *args, &blk)
      return self.[](method_name, &blk) if key?(method_name)
      match = method_name.to_s.match(/(.*?)([?=!]?)$/)
      case match[2]
      when "="
        self[match[1]] = args.first
        # ...
      else
        default(method_name, *args, &blk)
      end
    end

    # ...
  end
end
```

If the name of the called method is the name of a key in the hash (such as flavor), then Hashie::Mash#method_missing simply calls the [] method to return the corresponding value. If the name ends with a "=", then method_missing chops off the "=" at the end to get the attribute name and then stores its value. If the name of the called method doesn't match any of these cases, then method_missing just returns a default value. (Hashie::Mash also supports a few other special cases, such as methods ending in "?", that were scrapped from the code above.)